Desarrollo de sistema software

Proyecto: Sanos y Salvos

Sección: DSY1106-002D

Docente: Viviana Poblete
Integrantes: Daniel Onetto y Constanza Orellana

Índice

Contextualización del caso..... 2
Necesidades del Cliente..... 3
Diagrama de Patrón de Microservicios.....4
Justificación de herramientas y estrategias.....4
Esquema de Comunicación entre microservicios..... 5
Justificación de la Arquitectura..... 5
3. Flujo de Retroalimentación y Notificaciones.....6
Esquema de Arquitectura.....7
Evaluación general del diseño propuesto..... 7
Fortalezas..... 7
Debilidades o desafíos..... 7
Conclusión..... 8
Bibliografía.....9

Contextualización del caso

En los últimos años, el aumento sostenido en la tenencia de mascotas ha transformado la percepción social sobre su bienestar, posicionándose como miembros fundamentales del núcleo familiar. Sin embargo, en entornos urbanos densamente poblados, el extravío de animales se ha convertido en una problemática crítica y frecuente que afecta la salud emocional de los dueños y satura las capacidades de las organizaciones de rescate.

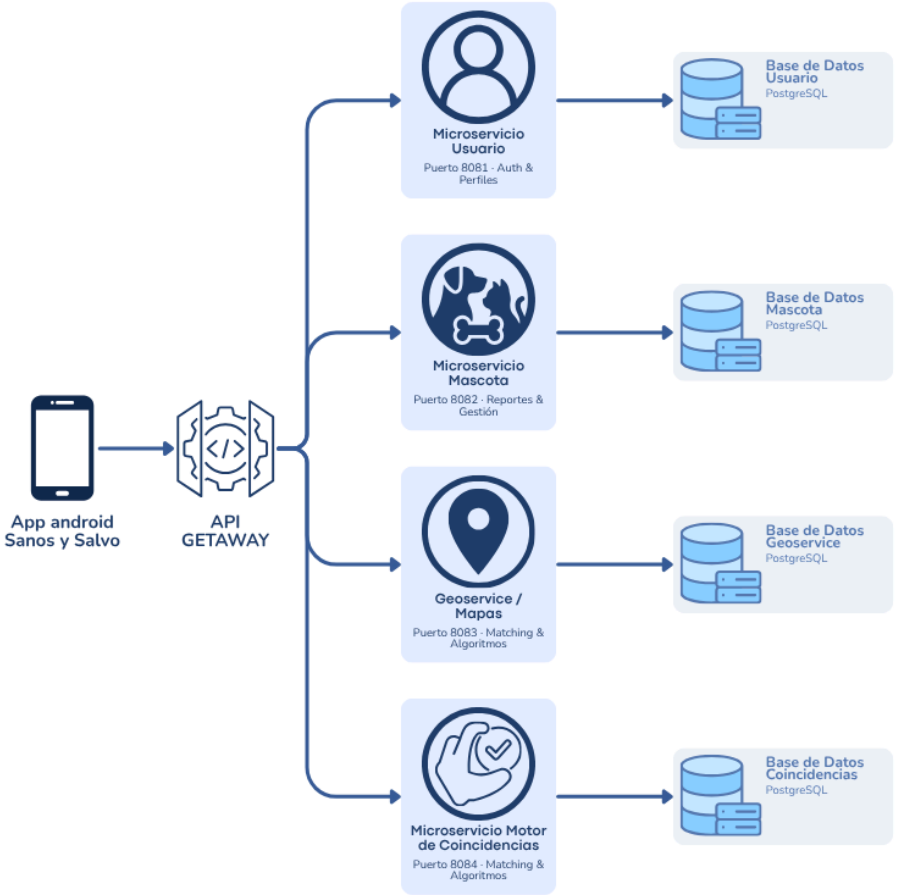
Actualmente, la respuesta ante un extravío es mayoritariamente reactiva y fragmentada; se depende de publicaciones masivas en redes sociales, el pegado de carteles físicos y el contacto aleatorio con refugios o clínicas. Esta dispersión de la información genera métodos desorganizados y lentos que reducen drásticamente las probabilidades de una recuperación exitosa. Ante este escenario, surge **"Sanos y Salvos"**, una plataforma tecnológica inteligente diseñada para centralizar la información y transformar la búsqueda pasiva en una red de colaboración activa y geolocalizada. A diferencia de los registros estáticos, esta solución propone un ecosistema digital dinámico que conecta en tiempo real a ciudadanos, clínicas veterinarias, refugios y municipalidades para optimizar la localización y el rescate de mascotas perdidas.

Necesidades del Cliente

El requerimiento central de **Sanos y Salvos** es la superación de la brecha informativa actual mediante un sistema unificado y automatizado que resuelva las siguientes necesidades específicas:

- **Centralización e Inteligencia de Datos:** Implementar una plataforma que consolide reportes de diversas fuentes —dueños, rescatistas, clínicas y entidades públicas— en una estructura de datos estandarizada que permita la interoperabilidad entre actores.
- **Visualización Geográfica Dinámica:** Desarrollar un sistema de geolocalización que permita a los usuarios visualizar en un mapa interactivo no solo los puntos exactos de extravío con radios de incidencia, sino también la ubicación de centros de ayuda, veterinarias y refugios cercanos.
- **Gestión de Reportes y Validación:** Facilitar el ingreso de reportes estructurados que incluyan características físicas detalladas, fotografías y datos de contacto, incorporando mecanismos para notificar encuentros y verificar la identidad de la mascota para asegurar que se trate del mismo ejemplar.
- **Automatización de Coincidencias:** Integrar un motor de búsqueda que utilice algoritmos avanzados para cruzar automáticamente los datos de mascotas perdidas y encontradas en función de raza, color, tamaño, ubicación geográfica y fecha, notificando de manera oportuna a las partes interesadas.
- **Escalabilidad y Experiencia de Usuario:** Garantizar que la solución sea una aplicación móvil intuitiva, responsiva y altamente escalable, capaz de manejar múltiples solicitudes concurrentes y de operar con alta disponibilidad para ofrecer una respuesta inmediata ante emergencias.

Diagrama de Patrón de Microservicios



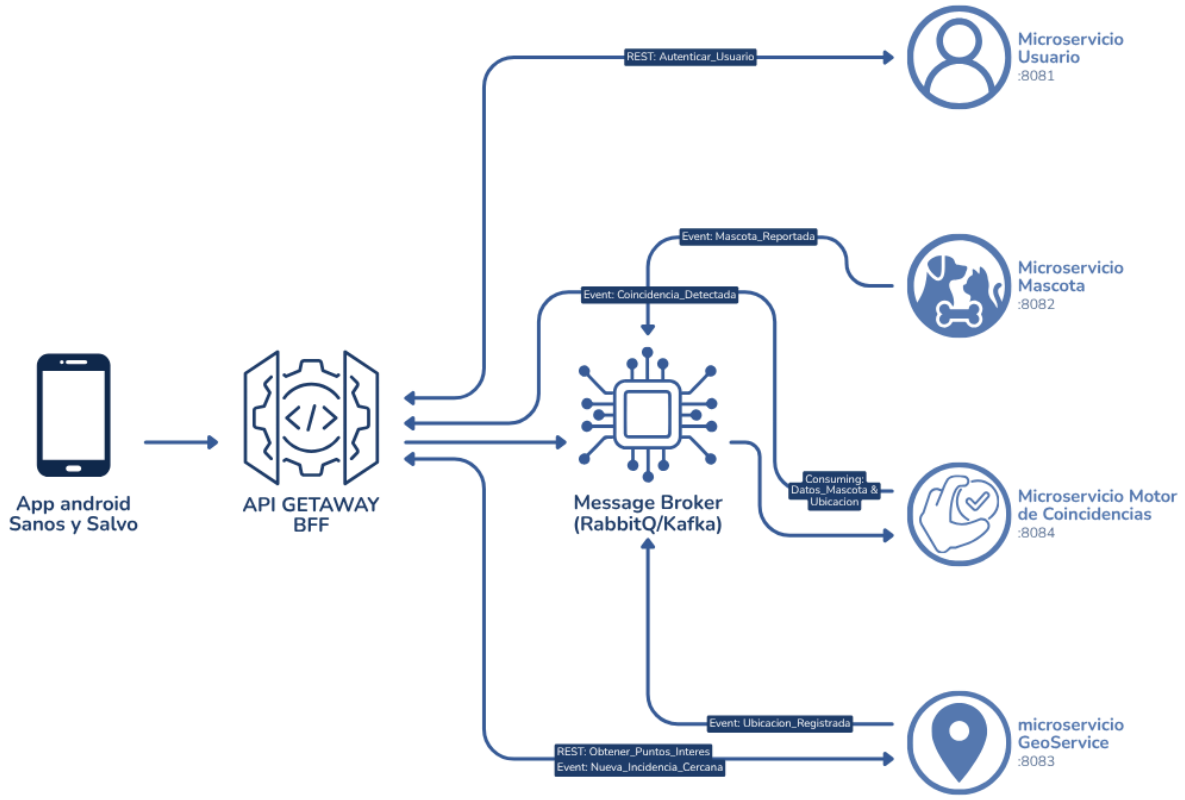
Justificación de herramientas y estrategias

El desarrollo del sistema **Sanos y Salvos** utiliza herramientas alineadas con los estándares de la industria para garantizar modularización y escalabilidad. Al integrar **Jetpack Compose** para el frontend móvil y una arquitectura de microservicios en el backend, se asegura una solución robusta capaz de manejar múltiples solicitudes concurrentes y flujos de datos en tiempo real provenientes de la red de colaboración ciudadana y geolocalización.

- **Jetpack Compose (Frontend Móvil):** Elegido por su naturaleza declarativa, permitiendo que la interfaz tipo "Waze" reaccione de forma fluida a los cambios de estado en el mapa, como la aparición de nuevos reportes o la actualización de radios de incidencia en tiempo real.
- **Kotlin & Retrofit:** Garantizan un código moderno y seguro, facilitando la comunicación eficiente entre la aplicación móvil y el **API Gateway** para el consumo de servicios REST de localización y gestión de usuarios.

- **Arquetipos Maven Personalizados:** Estandarizan la creación de los microservicios (**Usuarios, Mascotas, Geoservice y Coincidencias**), asegurando una estructura base coherente que facilita la integración de nuevas funcionalidades geográficas.
- **Spring Data JPA & Repository Pattern:** Optimizan la persistencia de datos de los reportes estructurados y desacoplan la lógica de negocio del acceso a la base de datos, otorgando la flexibilidad necesaria para manejar grandes volúmenes de información geográfica.
- **Circuit Breaker:** Estrategia de resiliencia vital que evita el colapso del sistema; por ejemplo, si el microservicio de mapas experimenta latencia, el patrón permite que las funciones de registro de mascotas y perfiles sigan operativas.
- **SonarQube & Git Flow:** Aseguran la calidad del software con una cobertura mínima del **60% del código** y un flujo de trabajo colaborativo organizado mediante ramas de desarrollo, fundamental para la integración constante de mejoras en el motor de coincidencias.

Esquema de Comunicación entre microservicios



Justificación de la Arquitectura

La arquitectura se basa en un modelo **híbrido** que combina peticiones síncronas para una respuesta inmediata al usuario y un modelo orientado a eventos (Event-Driven) para procesos pesados de análisis.

1. Comunicación Síncrona (App <-> Gateway <-> Microservicios)

Se utiliza el protocolo HTTPS/REST para acciones donde el usuario requiere una respuesta instantánea.

- **App ↔ API Gateway BFF:** Intercambio de tokens de seguridad y datos optimizados para la vista móvil.
- **API Gateway → M.S. Usuarios:** Consulta de perfil y validación de credenciales.
- **API Gateway → M.S. Geoservice:** Carga inicial de los puntos de interés (veterinarias, refugios) y reportes activos en el mapa según la ubicación actual del usuario.

2. Comunicación Asíncrona Orientada a Eventos (Message Broker)

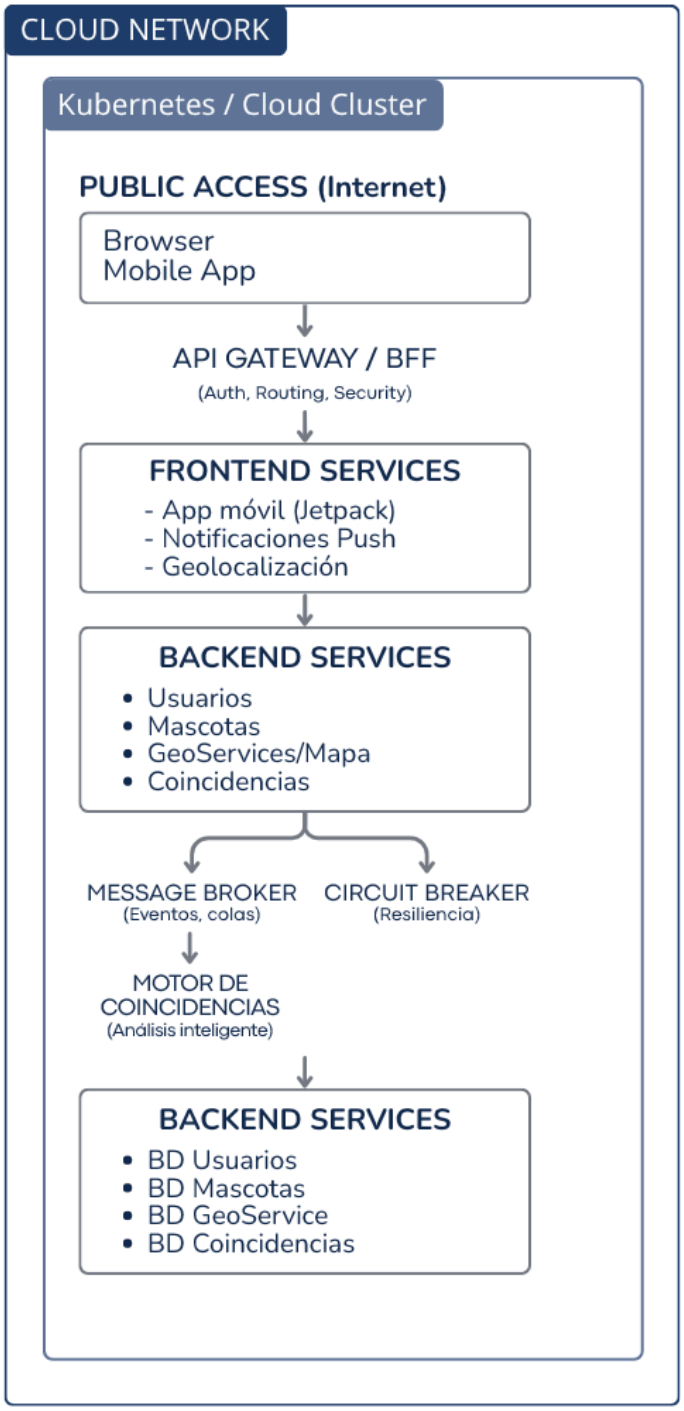
Para evitar que la App se bloquee mientras se procesan datos complejos, se utiliza el Message Broker como intermediario.

- **M.S. Mascotas → Message Broker:**
  - **Event: Mascota_Reportada:** Se dispara cuando un usuario registra un extravío o un avistamiento. Contiene características físicas y fotos.
- **M.S. Geoservice → Message Broker:**
  - **Event: Ubicacion_Actualizada:** Se dispara cuando se asocia una nueva coordenada geográfica a un reporte existente o nuevo.
- **Message Broker → M.S. Motor de Coincidencias:**
  - El motor consume ambos eventos de forma automática. Al recibir la información, inicia el algoritmo de cruce de datos (raza, color y cercanía geográfica) sin que el usuario tenga que esperar en la pantalla de carga.

3. Flujo de Retroalimentación y Notificaciones

- **M.S. Motor de Coincidencias → API Gateway:**
  - **Event: Coincidencia_Detectada:** Una vez que el algoritmo encuentra una posible pareja entre "Perdido" y "Encontrado", envía una señal al Gateway.
- **API Gateway → App Android:**
  - **Notificación Push:** El sistema rompe la unidireccionalidad para enviarle al usuario una alerta inmediata sobre el posible hallazgo de su mascota.

Esquema de Arquitectura



Evaluación general del diseño propuesto

El diseño propuesto responde de manera efectiva a los requerimientos del cliente tanto a nivel funcional como técnico.

Fortalezas	Debilidades o desafíos
Alta escalabilidad: Los microservicios permiten manejar un crecimiento en usuarios y datos sin afectar a todo el sistema. **Desacoplamiento:** La independencia entre servicios facilita el mantenimiento y evolución del sistema. **Procesamiento eficiente:** El uso de mensajería permite manejar grandes volúmenes de datos de rastreo sin sobrecargar el sistema. **Disponibilidad:** Estrategias como Circuit Breaker evitan caídas totales. **Experiencia de usuario:** La app móvil con datos en tiempo real mejora la interacción y utilidad del sistema.	**Mayor complejidad:** Comparado con un sistema monolítico, requiere mayor conocimiento técnico. **Gestión de infraestructura:** Necesita herramientas adicionales para monitoreo, despliegue y coordinación de servicios. **Consistencia de datos:** Puede ser más difícil mantener consistencia entre múltiples bases de datos.

Conclusión

El diseño arquitectónico presentado para la plataforma **"Sanos y Salvos"** marca una evolución crítica desde los métodos de búsqueda tradicionales y fragmentados hacia un ecosistema digital inteligente, colaborativo y de alta disponibilidad. Al adoptar un enfoque de **microservicios orientados a eventos**, se ha logrado construir una solución que prioriza la inmediatez y la precisión geográfica, elementos fundamentales en situaciones de emergencia donde cada minuto cuenta para la recuperación de una mascota.

La implementación estratégica de patrones como el **API Gateway/BFF** y el **Message Broker** permite que la aplicación móvil opere con la fluidez y eficiencia de herramientas líderes en geolocalización, garantizando que el **Motor de Coincidencias** procese grandes volúmenes de datos en segundo plano sin degradar la experiencia del usuario. Asimismo, el uso de contenedores **Docker** y el patrón de **Base de Datos por Servicio** (PostgreSQL) asegura que

el sistema sea escalable, resiliente y capaz de integrarse con actores clave como clínicas veterinarias y municipalidades.

En definitiva, esta arquitectura no solo cumple con los estándares técnicos de modularización y mantenibilidad exigidos en el ámbito del desarrollo **Fullstack**, sino que establece una base tecnológica sólida y sostenible para crear una red de seguridad ciudadana eficiente, transformando la angustia de la pérdida en una oportunidad de recuperación exitosa mediante la innovación tecnológica.

Bibliografía

- **Microservices Patterns:** Richardson, C. (2018). *Pattern: Database per Service*. microservices.io/patterns/data/database-per-service.html
- **API Gateway & BFF:** Microsoft Azure Architecture. *Backend for Frontends pattern*. learn.microsoft.com/en-us/azure/architecture/patterns/bff
- **Jetpack Compose:** Android Developers. *Compose para interfaces modernas*. developer.android.com/jetpack/compose
- **PostGIS (Geoservice):** PostGIS Project. *Spatial and Geographic Objects for PostgreSQL*. postgis.net/documentation/
- **Spring Data JPA:** VMware Tanzu. *Spring Data JPA Reference Guide*. spring.io/projects/spring-data-jpa
- **RabbitMQ (Message Broker):** VMware. *Messaging with RabbitMQ*. rabbitmq.com/tutorials
- **SonarQube Quality:** SonarSource. *Clean Code and Code Coverage*. docs.sonarsource.com/sonarqube/latest/